

eBook for Pizza.owl Ontology Practice

This is the eBook to describing the detail contents from the series demo videos.

Every chapter maps to the respective # of video.



Enjoy reading...

Here are the quick links to every chapter:

- [Chapter 01 - Entering the World of Ontology Engineering with Protégé and Pizza.owl](#)
- [Chapter 02 - Building Your First Ontology in Protégé \(WIP\)](#)

Chapter 01 - Entering the World of Ontology Engineering with Protégé and **Pizza.owl**

The journey into ontology engineering often begins with a deceptively simple example: **pizza**. Behind this famous **pizza.owl** tutorial lies one of the most influential learning paths in the Semantic Web and Knowledge Graph ecosystem. In this first chapter, we establish the conceptual foundations of ontology modeling, introduce the Protégé ontology editor, and position ontology engineering within the broader vision of Executable Knowledge Architecture (EKA).

This chapter is based primarily on Michael DeBellis' excellent tutorial [A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins](#), which modernized and extended the original Manchester Pizza Tutorial into a comprehensive hands-on learning experience.

At the same time, this book extends beyond the tutorial itself. The goal here is not only to learn Protégé as a standalone modeling tool, but to understand ontology as a critical layer in the transformation from diagrams to executable intelligence.

- [Chapter 01 - Entering the World of Ontology Engineering with Protégé and **Pizza.owl**](#)
 - [1.1 Why the Pizza Tutorial Became an Industry Classis](#)
 - [1.2 Protégé: More Than an Ontology Editor](#)
 - [1.3 The EKA Perspective: From Diagrams to Executable Intelligence](#)
 - [1.4 Understanding OWL: The Semantic Core](#)
 - [1.4.1 Classes: Modeling Conceptual Categories](#)
 - [1.4.2 Individuals: Representing Concrete Instances](#)
 - [1.4.3 Properties: Defining Relationships](#)
 - [1.5 Ontology vs Traditional Data Modeling](#)
 - [1.6 Open World Assumption: A New Way of Thinking](#)
 - [1.7 Reasoners: The Engine Behind Semantic Intelligence](#)
 - [1.8 Why Ontology Matters in the AI Era](#)
 - [1.9 What You Will Build Throughout This Book](#)
 - [1.10 The Bigger Picture](#)
- [1.11 Chapter 01 Summary](#)
 - [Next Chapter Preview](#)
 - [Reference](#)
 - [Demo Video for this Chapter](#)
 - [One More Thing...](#)

1.1 Why the Pizza Tutorial Became an Industry Classis

The **pizza.owl** tutorial has become the “Hello World” of ontology engineering for several reasons.

First, pizza is universally understandable. Nearly everyone intuitively understands toppings, ingredients, vegetarian classifications, spicy foods, and regional styles. This makes it easier to focus on semantic modeling concepts rather than domain complexity.

Second, the tutorial gradually introduces increasingly sophisticated OWL concepts in a controlled environment. Instead of overwhelming learners with abstract logic theory, it teaches ontology construction incrementally:

- Classes
- Subclasses
- Individuals
- Object Properties
- Restrictions
- Reasoners
- Logical Inference
- SWRL Rules
- SPARQL Queries
- SHACL Validation

This progression mirrors how real enterprise ontology projects evolve.

Third, the tutorial demonstrates something fundamentally important:

Ontologies are not just diagrams.
They are executable semantic systems.

This is the core idea that later becomes central in EKA.

1.2 Protégé: More Than an Ontology Editor

Protégé is often introduced simply as an ontology editor, but this description dramatically understates its importance.

Protégé is better understood as a **semantic modeling platform**.

It allows users to:

- Define domain concepts
- Encode formal semantics
- Build machine-readable knowledge structures
- Execute logical reasoning
- Validate constraints
- Query semantic relationships
- Prepare foundations for knowledge graphs

In traditional enterprise architecture tools, architects often create static diagrams that describe systems visually but lack executable semantics.

For example:

- UML diagrams describe structure
- BPMN diagrams describe processes
- ER diagrams describe data
- Capability maps describe organizations

However, these artifacts usually remain disconnected documentation.

Protégé introduces a fundamentally different paradigm.

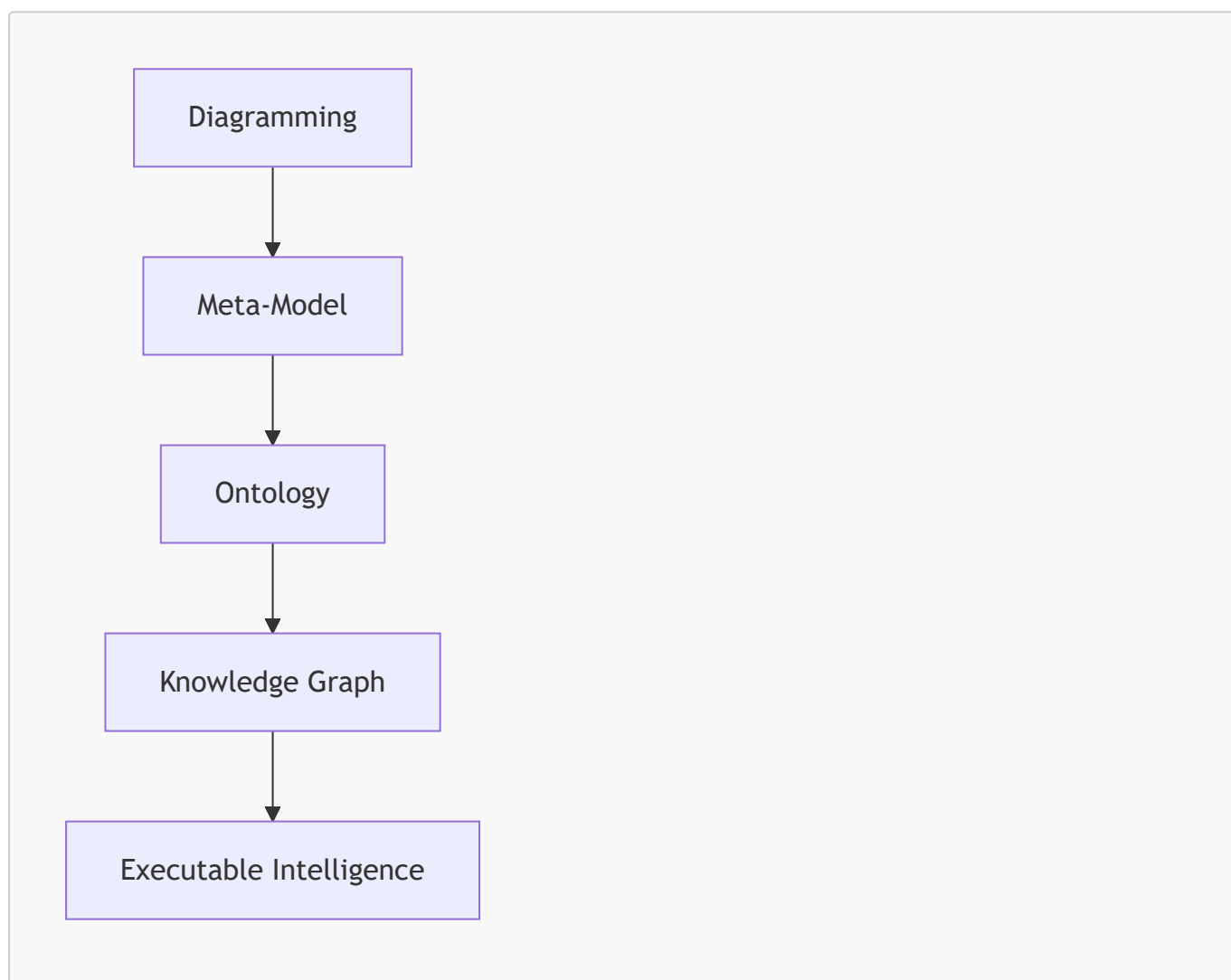
Instead of merely drawing concepts, we formally define them using **logic-based semantics**. The ontology becomes interpretable by machines, enabling automated reasoning and inference.

This transition is foundational for modern AI-native architectures.

1.3 The EKA Perspective: From Diagrams to Executable Intelligence

Within the EKA (Executable Knowledge Architecture) framework introduced on xiaoqi.com, ontology engineering occupies a critical transformation layer.

The implementation roadmap of EKA can be summarized as:



The Pizza OWL tutorial specifically addresses the "Ontology" stage.

To understanding why this matters, we must examine the limitations of traditional enterprise modeling.

Most enterprise architecture repositories contain thousands of diagrams:

- Application landscapes
- Process flows
- Integration maps

- Data models
- Organizational charts

But these assets - very valuable - are typically:

- Static
- Human-readable only
- Semantically ambiguous
- Non-executable
- Difficult to integrate

Ontology changes this situation fundamentally.

When enterprise concepts become ontological entities:

- Relationships gain formal meaning
- Constraints become machine-processable
- Inference becomes possible
- Knowledge graphs can be generated
- AI systems can consume structured semantics

In EKA terminology, ontology acts as the semantic bridge between abstract architecture and executable intelligence.

This is why learning Protégé is not merely learning another modeling tool.

IT IS learning how to formalize knowledge itself.

1.4 Understanding OWL: The Semantic Core

The tutorial centers around OWL, the Web Ontology Language.

[Web Ontology Language](#) is a W3C standard designed to represent knowledge in a machine-readable and logically rigorous form.

Unlike relational databases, OWL is not primarily designed for transaction processing.

Instead, OWL focuses on:

- Meaning
- Classification
- Inference
- Semantic relationships
- Knowledge interoperability

Michael DeBellis emphasizes that OWL ontologies are composed primarily of three core elements:

1. Classes
2. Properties
3. Individuals

These concepts form the semantic building blocks of all ontology systems.

1.4.1 Classes: Modeling Conceptual Categories

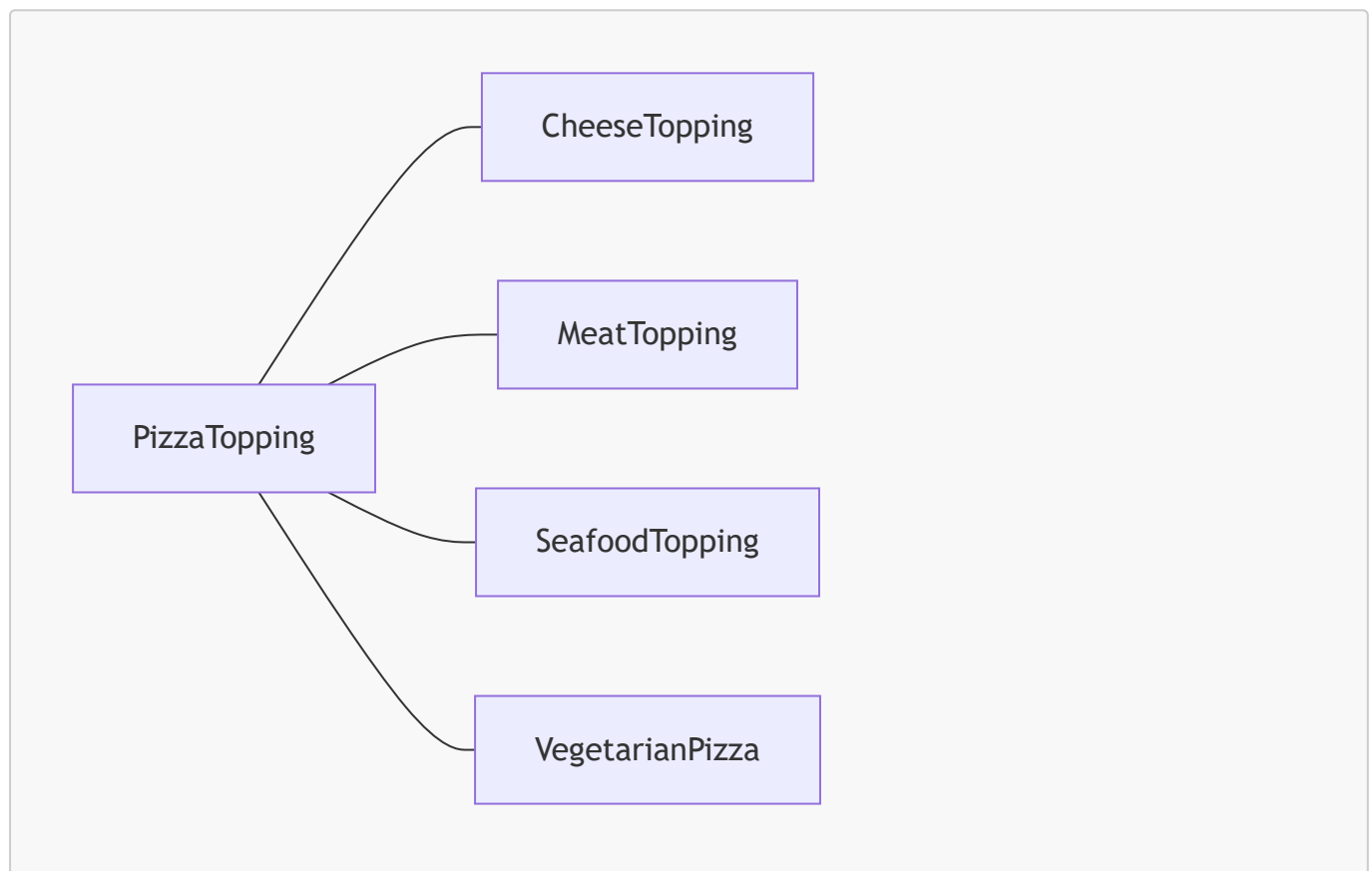
A class represents a category or type of thing.

Examples in the Pizza ontology include:

- Pizza
- PizzaTopping
- CheeseTopping
- VegetableTopping
- VegetarianPizza

Classes can be organized hierarchichally.

For example:



You may also document above graph in this way:

```
PizzaTopping
|-- CheeseTopping
|-- MeatTopping
|-- SeafoodTopping
|-- VegetableTopping
```

This resembles inheritance structure in object-oriented modeling, but OWL classes are fundamentally semantic rather than implementation-oriented.

A class in OWL describes a set of possible individuals.

This distinction becomes important later when reasoning engines classify instances automatically.

1.4.2 Individuals: Representing Concrete Instances

Individuals are actual members of classes.

For example, members of class `Pizza` or `PizzaTopping` can be:

- `MargheritaPizza`
- `MozzarellaTopping`
- `SpicyBeefPizza`

An individual belongs to one or more classes.

However, unlike traditional systems where classification is often manually assigned, OWL reasoners can infer class membership automatically base on logical definitions.

This capability is one of the major breakthroughs of ontology systems.

1.4.3 Properties: Defining Relationships

Properties define how entities relate to one another.

In OWL, there are two major property types:

1. Object Properties: These connect individuals to other individuals.

Example:

```
hasTopping
```

2. Data Properties: These connect individuals to literal value.

Example:

```
hasCalories  
hasPrice  
hasSpicinessLevel
```

Properties become extraordinarily powerful when combined restrictions and logical constraints.

1.5 Ontology vs Traditional Data Modeling

New learners often ask:

```
"How is ontology different from a database schema?"
```

This is one of the most important conceptual distinctions in semantic engineering.

Traditional databases focus on storing data efficiently.

Ontologies focus on representing meaning formally.

A relational schema might define:

```
Pizza(id, name)
ToppingI(id, name)
PizzaTopping(pizza_id, topping_id)
```

But the schema itself does not formally express semantic truths such as:

- Vegetarian pizzas cannot contain meat toppings
- Spicy pizzas contain spicy ingredients
- Certain toppings are subclasses of vegetable toppings
- Seafood toppings are disjoint from meat toppings

OWL allows these semantics to be explicitly modeled.

This is why ontologies become foundational for AI and knowledge graph systems.

1.6 Open World Assumption: A New Way of Thinking

One of the most difficult concepts for beginners is the Open World Assumption (OWA).

Traditional databases generally operate under a Closed World Assumption (CWA):

If something is not stored, it is considered false.

OWL works differently!

Under OWA:

Absence of knowledge does not imply falsehood.
It just means the knowledge that is unknown yet.

For example:

If an ontology does not specify whether a pizza contains meat, we cannot conclude that it is vegetarian.

This principle aligns much more closely with real-world knowledge representation, where information is often incomplete.

Understanding OWA is essential because it fundamentally changes how semantic systems behave compared to traditional enterprise systems.

Michael DeBellis highlights this repeatedly throughout the tutorial because many modeling errors originate from misunderstanding **open-world reasoning**.

1.7 Reasoners: The Engine Behind Semantic Intelligence

The real magic of ontology engineering appears when we introduce reasoners.

A reasoner is a logic engine that analyzes ontology axioms and derives new knowledge automatically.

For example, if we define:

- `VegetarianPizza` $\equiv$ `Pizza` AND NOT (`hasTopping` some `MeatTopping`)

and later create a pizza that only contains cheese and vegetables, the reasoner can automatically classify it as a `VegetarianPizza`.

No manual tagging is required!

This is a profound shift.

The ontology stops being passive documentation and becomes an executable semantic system.

This is precisely the transition EKA describes as moving toward executable intelligence.

1.8 Why Ontology Matters in the AI Era

Modern AI systems increasingly require structured semantic context.

Large Language Models (LLMs) are powerful, but without explicit knowledge structures they often struggle with:

- Consistency
- Explainability
- Governance
- Enterprise semantics
- Controlled reasoning

Poor structures for LLMs normally lead to the "beautiful garbage" to be generated by AI.

Ontology provides the semantic backbone that complements probabilistic AI systems.

This is why ontology engineering is re-emerging as a critical discipline in:

- Knowledge Graphs
- Semantic Search
- AI Agents
- Digital Twins
- Enterprise Knowledge Management
- Data Fabric architectures

The Pizza tutorial may appear simple, but the underlying modeling principles scale into enterprise-grade semantic systems.

1.9 What You Will Build Throughout This Book

As this eBook progresses, we will gradually build increasingly sophisticated ontology capabilities using Protégé.

The journey includes:

- Constructing class hierarchies
- Modeling object properties

- Defining logical restrictions
- Using automated reasoners
- Performing semantic queries
- Applying SWRL rules
- Validating ontologies with SHACL
- Connecting ontology concepts to knowledge graph thinking
- Positioning ontology inside the EKA execution pipeline

By the end, readers should understand not only how to use Protégé, but why ontology engineering matters strategically for modern AI-native enterprises.

1.10 The Bigger Picture

The Pizza ontology is not really about pizza.

It is about learning how to formalize knowledge.

Once you understand:

- classes,
- relationships,
- restrictions,
- inference,
- and semantic reasoning

you can model almost anything:

- enterprise architectures
- business capabilities
- manufacturing systems
- healthcare vocabularies
- supply chains
- digital twins
- AI agent memory systems
- and knowledge graphs

See https://github.com/yasenstar/ArchiMate_Ontology as one practical sample that I'm applying to use Protégé to build ontology for ArchiMate (EA Modeling language).

This is where ontology engineering evolves from academic theory into enterprise intelligence infrastructure.

The future of architecture is not static diagrams.

The future is executable semantics.

And Protégé is one of the gateways into that world.

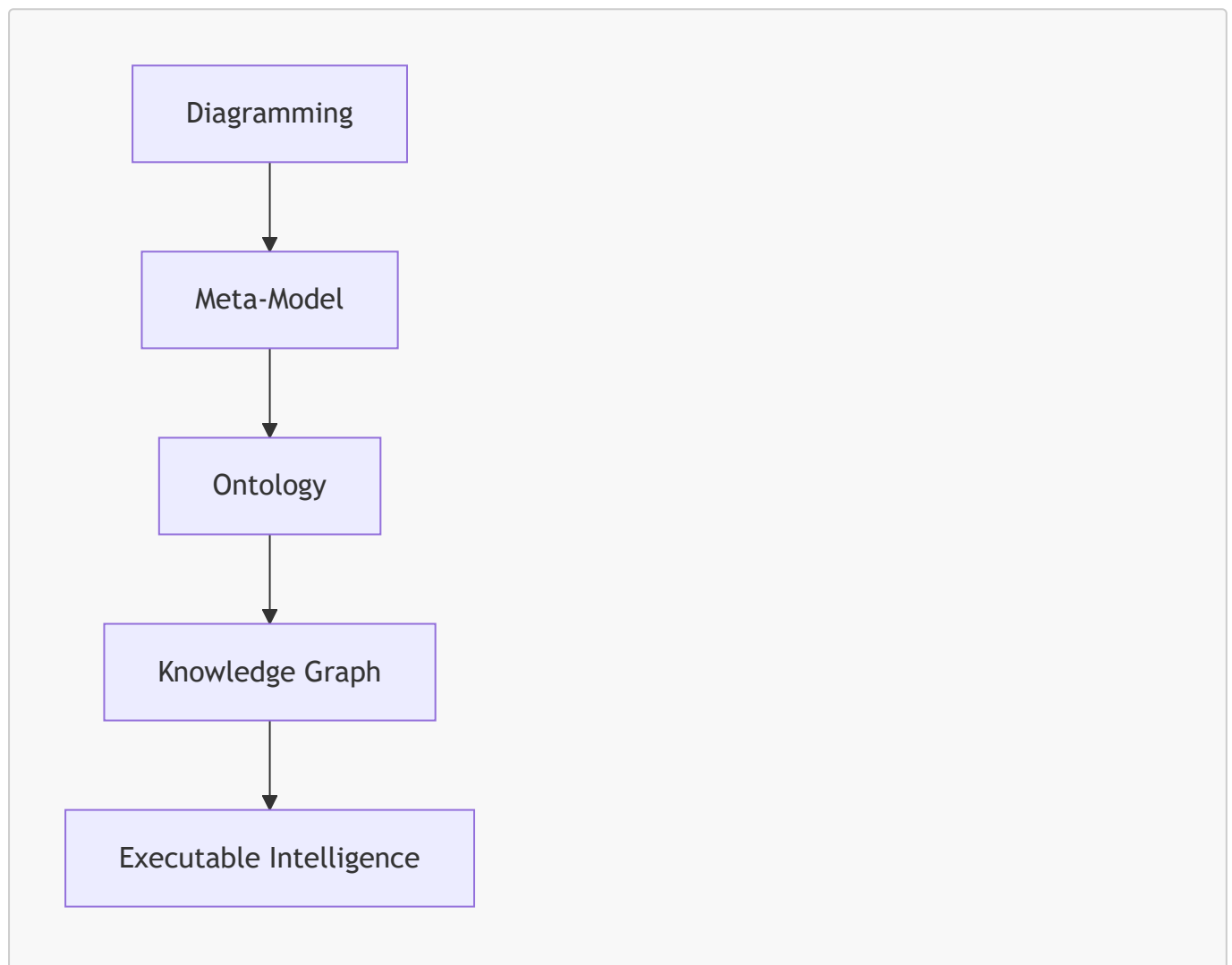
1.11 Chapter 01 Summary

In this chapter, we established the conceptual and strategic foundations of ontology engineering using the famous `pizza.owl` tutorial as the entry point. Rather than treating Protégé as merely a standalone modeling application (ontology editor), we positioned it as a semantic engineering platform capable of transforming static knowledge into executable intelligence.

We explored why the Pizza Tutorial became one of the most influential ontology learning examples in the Semantic Web community and examined how Protégé enables users to formally define concepts, relationships, restrictions, and semantic logic through OWL ontologies.

The chapter also introduced the EKA (Executable Knowledge Architecture) perspective, where ontology acts as the semantic bridge between enterprise modeling and knowledge-driven intelligence systems.

Within the EKA implementation roadmap:



the Pizza OWL tutorial specifically focuses on the ontology layer, helping learners understand how formal semantic structures are created and reasoned upon.

In addition, we introduced several core OWL concepts including:

- Classes
- Individuals
- Properties
- Open World Assumption

- Reasoners
- Semantic Inference

These concepts form the foundation for all future ontology engineering activities throughout this book.

Finally, we connected ontology engineering to the broader AI era, explaining why semantic modeling, knowledge graphs, and executable semantics are becoming increasingly important in modern enterprise architecture and intelligence systems.

Next Chapter Preview

In the next chapter, we will begin working directly inside Protégé and start constructing the foundational structure of the Pizza ontology.

Readers will learn how to:

- Create ontology projects
- Understand Protégé workspace components
- Build class hierarchies
- Create subclasses
- Define semantic categories
- Organize ontology structures properly

This marks the beginning of hands-on ontology engineering using Protégé.

The transition from theory to implementation starts there.

Reference

- Michael DeBellis, A Practical Guide to Building OWL Ontologies Using Protégé 5.5 and Plugins
- Protégé official resources and tutorials
- Protégé Pizza Ontology learning repository: https://github.com/yasenstar/protege_pizza
- EKA Official Website: <https://xiaoqi.com->

Demo Video for this Chapter

Visit <https://youtu.be/lOPZhqmTwfM> for watching this video in YouTube.

One More Thing...

Small tip, how to type **Protégé** in Markdown with those French **é**?

Answer: 😊, just using your keyboard's number pad, hold **Alt** key and key in **0233** or **233**, like **Alt + 0233** or **Alt + 233**, then you see the magic. (This is called Alt Code.) Enjoy!

Last updated at: 2026/05/11

Chapter 02 - Building Your First Ontology in Protégé

After establishing the conceptual foundations of ontology engineering in Chapter 01, we now move into the practical world of ontology construction inside Protégé. This chapter marks the transition from semantic theory into hands-on ontology modeling.

The goal of this chapter is not merely to "click through a tool," but to understand how semantic structures are engineered systematically. In traditional software engineering, developers write executable code. In ontology engineering, architects build executable meaning.

This distinction is fundamental.

The Pizza ontology may appear simple on the surface, but it introduces modeling patterns used throughout enterprise semantic systems, knowledge graphs, and AI-driven architectures.

This chapter is primarily based on the ontology construction sections from Michael DeBellis' revised [Pizza Tutorial for Protégé 5.5](#).

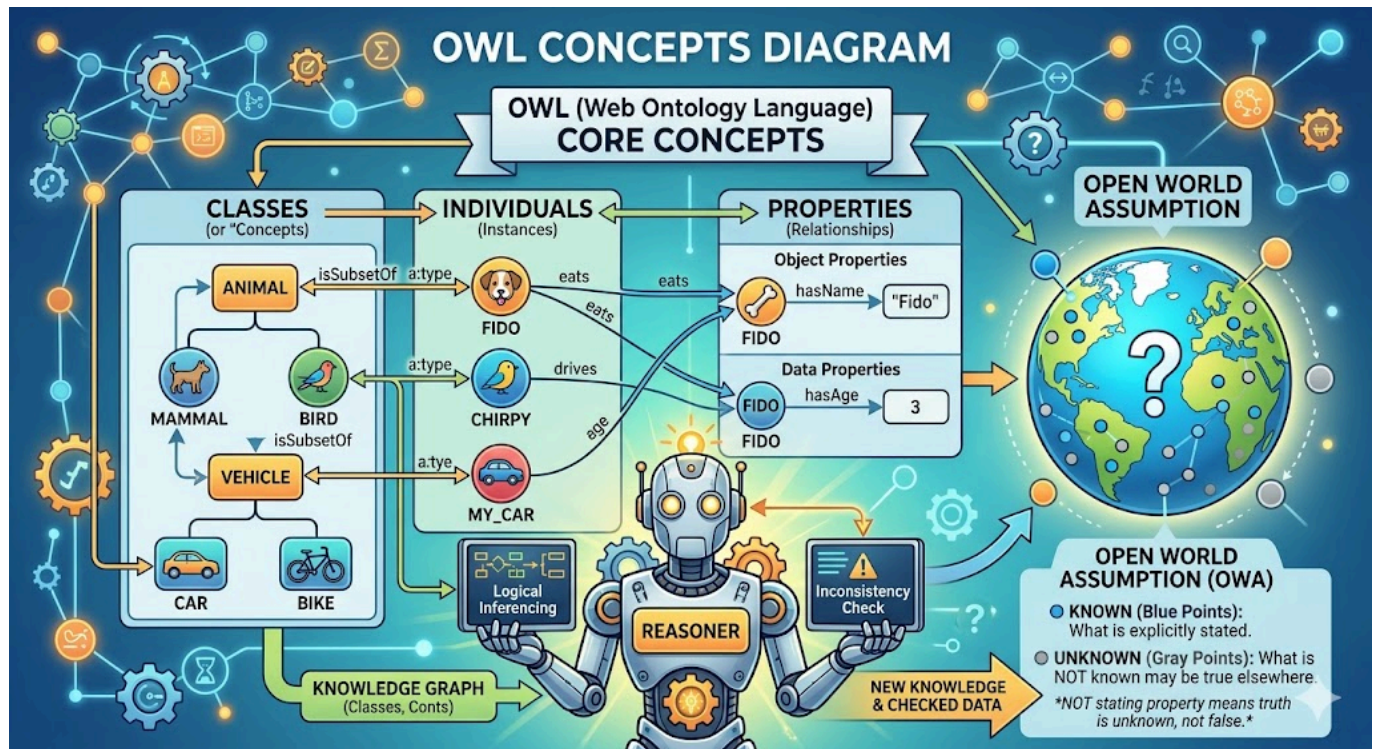
- [Chapter 02 - Building Your First Ontology in Protégé](#)
 - [2.1 From Theory to Construction](#)
 - [2.2 Understanding the Protégé Workspace](#)
 - [2.3 Creating a New OWL Ontology](#)
 - [2.4 The Importance of Ontology Naming Conventions](#)
 - [2.5 Understanding **Thing**: The Root of All Classes](#)
 - [2.6 Creating the Core **Pizza** Taxonomy](#)
 - [Reference](#)

2.1 From Theory to Construction

In Chapter 01, we introduced the core OWL concepts:

- Classes
- Individuals
- Properties
- Reasoners
- Open World Assumption

An quick AI-generated picture below brings them together systematically:



Now we begin building actual semantic structures.

This is where ontology engineering starts to feel very different from traditional modeling disciplines.

In a traditional diagramming tool, creating a class hierarchy is often just a visual activity. In Protégé, however, every modeling action contributes to a machine-processable semantic structure.

This means that:

- class hierarchies have logical implications
- property definitions affect reasoning behavior
- restrictions become executable constraints, and
- inference engines can derive new knowledge automatically

As Michael DeBellis emphasizes throughout the tutorial, ontology engineering should be approached as semantic knowledge modeling rather than merely graphical editing.

2.2 Understanding the Protégé Workspace

When Protégé is launched for the first time, many new users feel overwhelmed by the interface.

This is normal.

Protégé is not designed as a lightweight diagramming application. On the contrary, it is a **semantic engineering environment**.

The workspace contains multiple perspectives for editing different ontology components:

- Classes
- Object Properties
- Data Properties
- Individuals

- Annotations
- Reasoner views
- Query views
- Rule views

One of the most important concepts for beginners to understand is that Protégé does not separate "diagramming" from "logic".

Everything visible in the interface ultimately contributes to OWL axioms underneath.

This is why ontology engineering feels closer to knowledge programming than traditional modeling.

2.3 Creating a New OWL Ontology

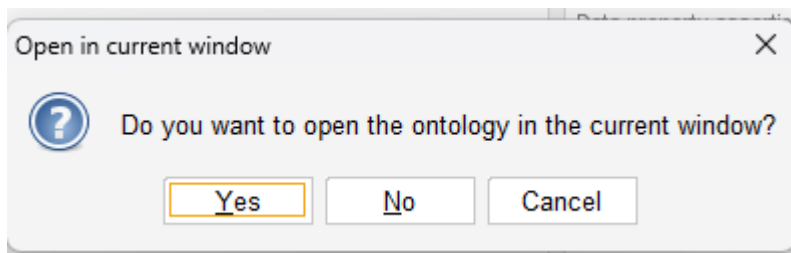
The first practical step is creating a new ontology project.

During this chapter's writing, I'm using Protégé v5.6.9.

Inside Protégé: **File** > **New...**:



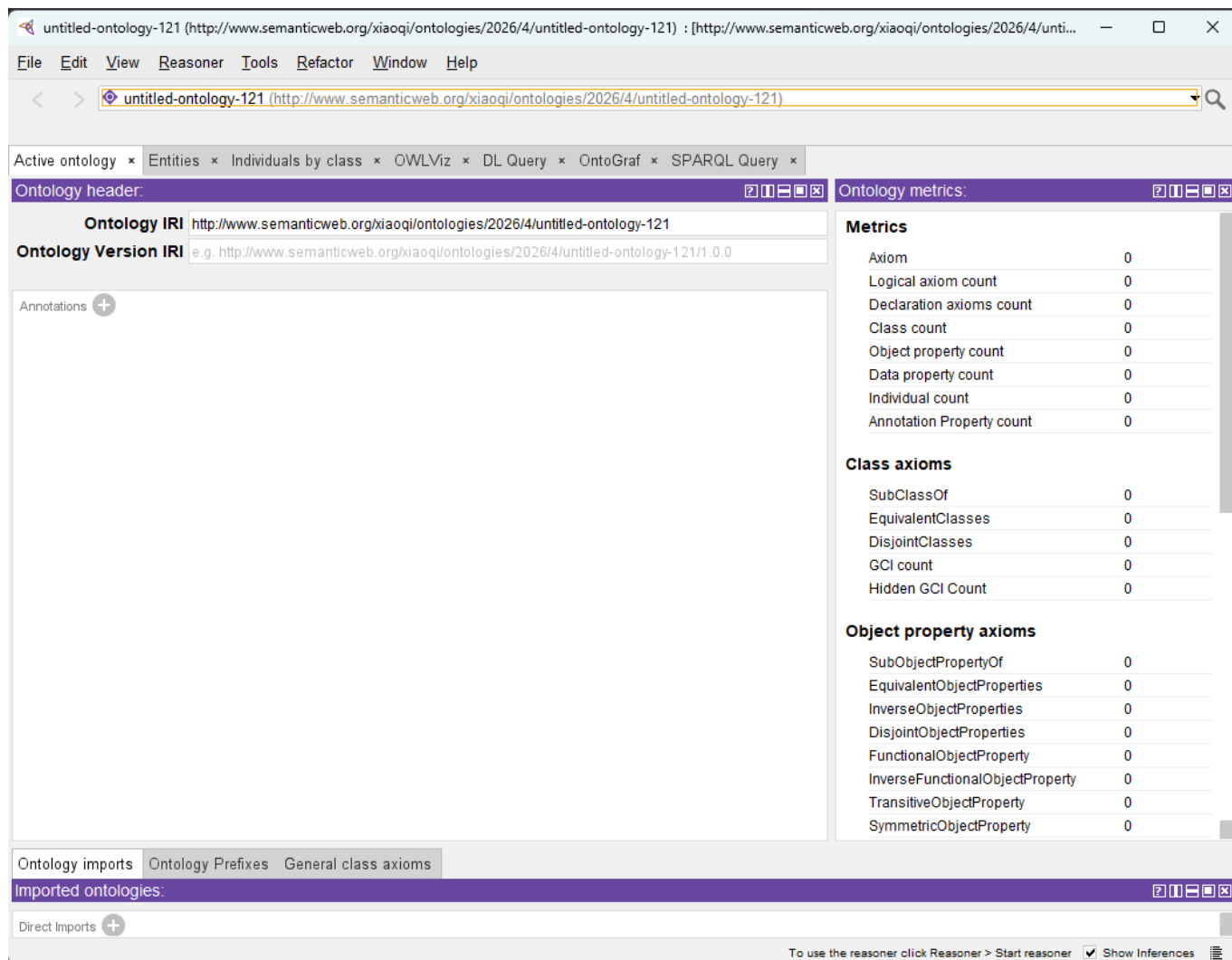
If you're having existing ontology file opened in Protégé, you will get below pop-up with question:



If you choose **Yes** (default), your current ontology file will be closed and your new ontology project will occupy current window, ensure you save any previous changes before losing them.

Or, you may choose **No** so a new window will be opened for your new ontology project, your existing opened ontology will not be impacted.

You then can see the new ontology project screen:



When you want to save the project, Protégé then asks for:

- Ontology IRI
- Document storage location
- RDF/XML or OWL serialization format

At this stage, beginners often underestimate the importance of ontology [IRIs - Internationalized Resource Identifier](#).

As defined by W3C, "Each ontology may have an ontology IRI, which is used to identify an ontology." An ontology IRI acts as the global semantic identifier for the ontology.

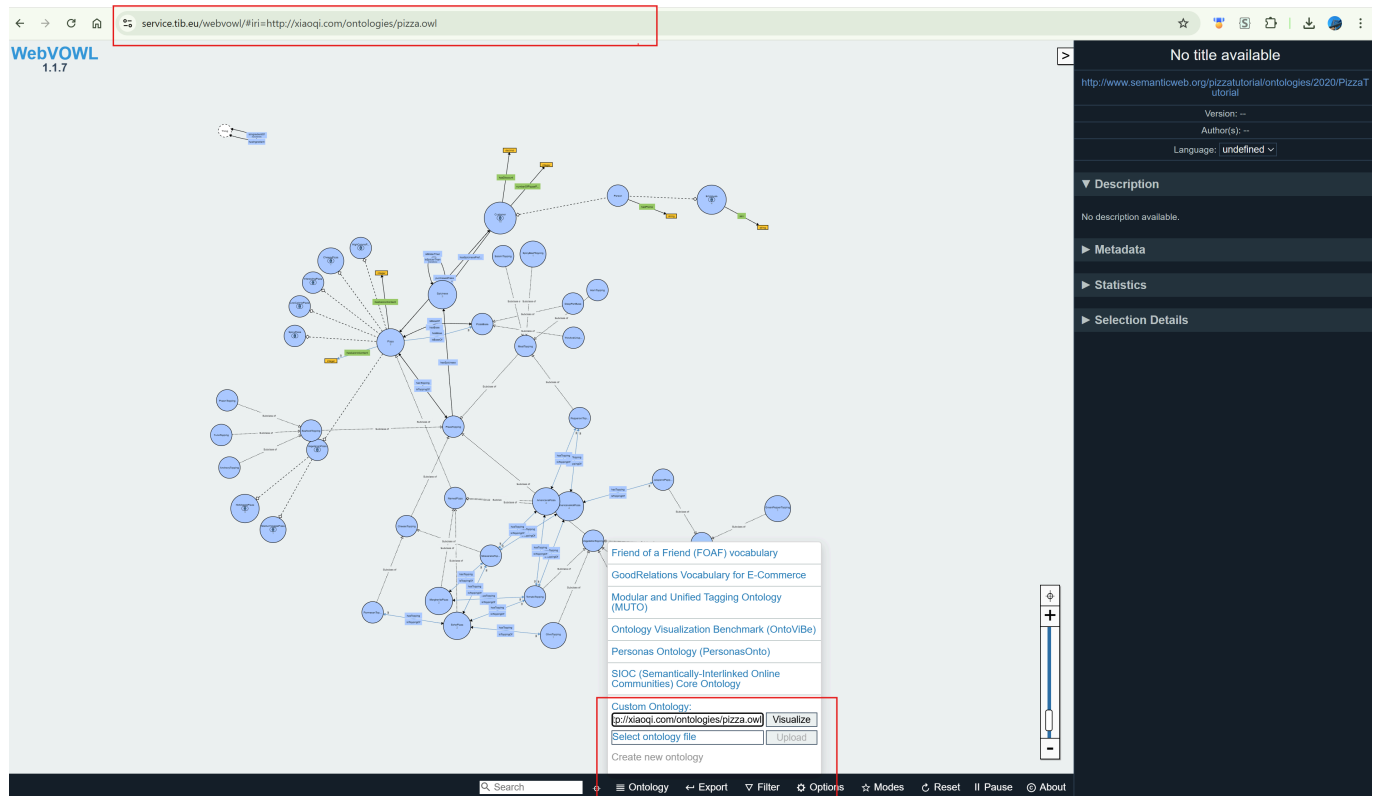
Unlike filenames or database table names, ontology IRIs are intended to be globally unique identifiers in semantic ecosystems.

For example:

```
# IRI for pizza.owl
http://xiaoqi.com/ontologies/pizza.owl

# My new ontology project IRI (local)
http://www.semanticweb.org/xiaoqi/ontologies/2026/4/untitled-ontology-121
```


Using [WebVOWL - the OWL Visualization Tool](#), you can paste above `pizza.owl` IRI then visualize it for now:



This is conceptually similar to namespaces in programming languages, but with web-scale semantic interoperability.

In enterprise environments, ontology naming strategies become critically important because ontologies may later integrate across:

- business domains
- data catalogs
- APIs
- knowledge graphs
- AI agents
- and semantic search platforms

This is one of the first points where ontology engineering begins to intersect with enterprise architecture governance.

2.4 The Importance of Ontology Naming Conventions

One of the subtle but important lessons from the Pizza tutorial is disciplined naming.

Ontology projects quickly become unmanageable if naming conventions are inconsistent.

Michael DeBellis repeatedly emphasizes semantic clarity and hierarchy organization throughout the tutorial.

Good ontology naming should satisfy several criteria, as below:

- Human-readable
- Machine-processable
- Semantically meaningful

- Consistent across domains
- Stable over time

For example in our `pizza.owl`:

```
Pizza
PizzaTopping
VegetarianPizza
SpicyPizza
```

These names are concise and semantically clear.

In enterprise ontology projects, naming conventions become even more important because ontologies may evolve into shared organizational semantic standards.

Within EKA, naming discipline is essential because ontology entities later become:

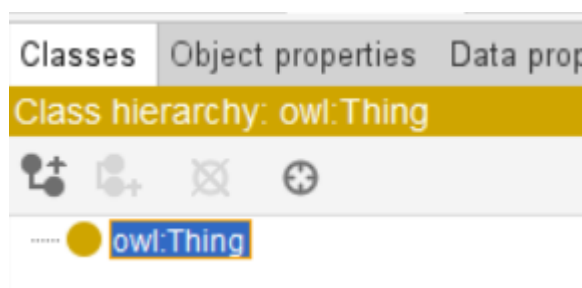
- graph nodes
- semantic APIs
- AI context objects
- digital twin entities
- and executable business semantics

Poor naming eventually creates **semantic debt** and chaos across the organization.

This is analogous to **technical debt** in software engineering.

2.5 Understanding `Thing`: The Root of All Classes

Every OWL ontology begins with a universal root class: `owl:Thing`



All classes ultimately inherit from `Thing`.

This concept initially appears trivial, but it represents an important philosophical principle in ontology engineering.

OWL assumes that all modeled entities exist within a universal semantic space.

For example:

```
Thing
|-- Pizza
```

```
| -- PizzaTopping  
| -- PizzaBase  
| -- Spiciness
```

Unlike relational databases where tables are independent structures, OWL organizes concepts into a globally connected semantic universe.

This is one reason ontology systems integrate naturally with knowledge graphs.

2.6 Creating the Core **Pizza** Taxonomy

The next step involves building the foundational class hierarchy.

From Wikipedia, **Taxonomy** is a practice and science concerned with classification or categorization.

The Pizza ontology begins with several top-level conceptual categories:

```
Pizza  
PizzaTopping  
PizzaBase  
Spiciness
```

These represent the primary semantic domains within the ontology.

At this stage, ontology engineering resembles taxonomy construction.

However, ontologies extend far beyond simple taxonomies.

An ontology additionally defines:

- semantics
- constraints
- properties
- logical restrictions
- inference behavior
- and reasoning rules

This distinction is critical.

Many enterprise "ontologies" are actually only taxonomies because they lack formal semantics.

Reference

- WebVOWL project in GitHub: <https://github.com/VisualDataWeb/WebVOWL>